

DATA ENGINEERING / OPEN SOURCE

Data Pipelines in Natural Language

User & operator manual

Version 0.1.0 | September 2026

Describe the work. Review the specification.
Validate the behavior. Deploy with evidence.

A practical guide to installation, catalogs, model configuration, pipeline authoring, quality controls, approval, Airflow deployment, evaluation, and troubleshooting.

Contents

1. What the MVP does
2. Install
3. First pipeline in five steps
4. Register assets
5. Write requests and handle clarification
6. Configure a model
7. Understand and modify the IR
8. Quality, quarantine, retries, and notifications
9. Review, approve, and deploy
10. Observe and propose repairs
11. Evaluate and test
12. Troubleshooting and operational limits

Read the release validation report alongside this manual. Offline corpus scores, mocked-provider tests, real-model checks, and Airflow execution establish different kinds of evidence.

1. What the MVP does

nlpipe turns a request into a typed Pipeline IR, resolves registered assets and capabilities, inserts quality barriers, validates operational and governance requirements, and deterministically generates an Apache Airflow 3 DAG. The IR is the source of truth. Edit or regenerate the IR; do not edit generated Python.

The reference runtime processes bounded, in-memory tables. It supports CSV, JSON, JSONL, Parquet, registered SQLite tables, and explicitly enabled HTTPS JSON reads. Transformations include filtering, field normalization, projection/renaming, deduplication, aggregation, joins, unions, and lookup enrichment. Quality checks include schema, required values, uniqueness, numeric range, freshness, and row count. Quarantine separates rejected records before downstream processing. Notifications produce local outbox events.

The registry also declares extension interfaces for PostgreSQL, object storage, API output, Python callables, SQL transformations, embeddings, entity resolution, model enrichment, anomaly detection, distribution checks, and arbitrary branching. These entries are unavailable until implemented and approved. They do not generate placeholder executable tasks. SQLite is the implemented generic SQL adapter. A checkpoint checks the number of supplied upstream datasets; it is not a file-arrival sensor.

Two frontends share the same validation boundary. The default offline frontend accepts a limited, documented vocabulary. An OpenAI-compatible frontend accepts richer requests using a locally hosted or explicitly authorized remote model. Model output remains untrusted and cannot grant approval. Neither frontend guarantees that a plausible pipeline captures every business requirement: review the IR and execute representative fixtures.

2. Install

Use Python 3.11 or newer. Create a virtual environment before installation.

```
git clone https://github.com/drcrael/data-pipelines-in-natural-language.git
cd data-pipelines-in-natural-language
python -m venv .venv
source .venv/bin/activate
python -m pip install ".[dev,parquet]"
nlpipe --help
```

On Windows PowerShell, activate with `.venv\Scripts\Activate.ps1`. Use python commands directly; the Python CLI is portable. Airflow execution is tested on Linux. A Windows or macOS workstation can author, validate, compile, and locally test pipelines, then deploy to a Linux Airflow environment.

Use a Python 3.12 environment for the tested Airflow runtime below. Airflow 3.1.8 requires Python below 3.14; the base CLI also supports Python 3.14.

```
python -m pip install "apache-airflow==3.1.8"
```

Install `nlpipe` and its relevant extras in both the DAG processor and every worker environment. Airflow must be configured separately, including its database, executor, authentication, and scheduler. This project does not provision a cluster. Release wheels and source archives can be installed directly with `python -m pip install PATH_TO_ARTIFACT`.

3. First pipeline in five steps

The repository contains synthetic orders and customers in `examples/fixtures`. The catalog describes where those assets live relative to an explicitly selected data root.

```
nlpipeline create "Load orders into analytics daily at 2 AM." \
  --catalog examples/catalog.yaml --out work/first.yaml
nlpipeline explain work/first.yaml
nlpipeline validate work/first.yaml --catalog examples/catalog.yaml
nlpipeline test work/first.yaml --catalog examples/catalog.yaml \
  --airflow --fixture-root examples/fixtures
nlpipeline compile work/first.yaml --catalog examples/catalog.yaml \
  --out work/first.py
```

`create` displays the interpreted intent, IR, validation decisions, and DAG location. It writes a YAML specification and a neighboring Python DAG. It does not deploy. The output path must be new so an accidental rerun cannot replace a reviewed pipeline.

`test --fixture-root` copies the fixture tree into a temporary sandbox. It runs the pipeline there and removes the sandbox afterward. Your fixtures remain unchanged. `--airflow` adds real DAG import and graph checks; without it, the output explicitly says that the Airflow gate was not requested.

To inspect persistent output, copy the fixtures to a working directory and run locally:

```
python -c "import shutil; shutil.copytree('examples/fixtures', 'work/demo-data')"
nlpipeline run examples/orders.yaml --catalog examples/catalog.yaml work/demo-data
nlpipeline status orders_daily work/demo-data
nlpipeline ask orders_daily "Why did the last run fail?" work/demo-data
```

The supplied orders example reads four rows, deduplicates by `order_id` to three, quarantines one record without `customer_id`, and writes two valid rows. Outputs appear under `work/demo-data/output`; evidence appears under `work/demo-data/.nlpipeline/runs/orders_daily`. Replacing an output is an intentional write. Use dedicated working data, not irreplaceable originals.

4. Register assets

The catalog is an administrator-owned YAML or JSON file. Requests reference identifiers; models never supply physical paths or credentials. Paths are relative to the runtime data root. Absolute paths, traversal, and symlinks escaping that root are rejected.

assets:

- identifier: orders
 type: csv
 location: orders.csv
 schema:
 order_id: integer
 customer_id: integer
 amount: number
 region: string
 allowed_operations: [read]
- identifier: analytics
 type: jsonl
 location: output/analytics.jsonl
 schema: {}
 allowed_operations: [write]

Identifiers use lowercase letters, digits, and underscores and begin with a letter. Assets can also contain owner, tags, aliases, description, freshness metadata, quality metadata, lineage, a registered SQL table, an external-write flag, and a secret reference.

An alias may deliberately match more than one asset. Ambiguity produces a question. Do not make an arbitrary destination the default simply to avoid clarification. Supply a registered ID or revise the catalog.

SQLite declarations require a safe table identifier. Reads use a fixed generated SELECT and bound limit. Writes bind every row value. The adapter owns the destination table: replacement recreates it using the registered schema. Do not point it at a table with unmanaged constraints, indexes, triggers, or application dependencies.

Catalog changes invalidate existing approvals and compiled catalog hashes. Revalidate and recompile after a change. The catalog, runtime root, and signing-key access must be protected by operating-system permissions.

5. Write requests and handle clarification

Useful offline examples include:

Load orders into analytics daily at 2 AM.
Read orders, deduplicate using order_id, and write analytics daily at 2 AM.
Load orders into analytics and validate schema.
Load orders into analytics; quarantine records without customer_id.
Read customers, normalize name strip, and write analytics.
Load orders; aggregate sum amount by region into analytics.
Load orders into analytics every day except Sunday.
Load orders into analytics and parquet_store.

Use explicit asset IDs, column names, operations, and run times. The offline grammar is conservative about missing configuration but is not a general language-understanding system. Use reviewed IR or the model frontend for requirements outside these patterns. Always inspect parameters and graph edges; successful compilation alone is not semantic acceptance.

A clarification result has status needs_clarification and questions containing a field, question, and reason. It exits with code 3 and emits no executable artifact. A rejected request exits with code 2. Do not repeatedly retry an ambiguous request unchanged: supply the missing source, destination, keys, thresholds, adapter, or exact schedule.

Requests involving yesterday's records need a timestamp field and interval semantics. The IR supports window: previous_day with an explicit incremental_field; the runtime interprets calendar boundaries in the pipeline timezone. Persistent high-water-mark incremental processing requires a separate adapter and is rejected in this MVP.

6. Configure a model

A local OpenAI-compatible service can be configured without a cloud account:

```
base_url: http://127.0.0.1:11434/v1
model: YOUR_INSTALLED_MODEL
json_schema: true
timeout_seconds: 900
allow_remote_metadata: false
```

The CPU audit uses a 16,384-token Ollama context and a 900-second request budget. Configure the server context accordingly; for a new CLI server, use `OLLAMA_CONTEXT_LENGTH=16384 ollama serve`. Consult the validation report for measured model results.

Save this as `work/local-model.yaml` and run:

```
nlpipe create "Read orders and write analytics on demand." \
  --catalog examples/catalog.yaml --config work/local-model.yaml \
  --out work/model-pipeline.yaml
```

Literal loopback addresses are local. `localhost` and LAN hostnames are not automatically trusted as local. Remote endpoints require HTTPS and explicit `allow_remote_metadata: true`. The model receives the request, asset IDs/types/schemas/aliases/allowed operations, capability contracts, response schema, and previous IR for modifications. It does not receive source rows, catalog paths, or credential values. Request text and metadata can still be sensitive; choose an appropriate provider.

For authentication, set `api_key_env` to an environment-variable name in the provider file and export its value separately. Do not put a credential in a prompt, IR, endpoint URL, or generated DAG. HTTP redirects and environment proxy inheritance are disabled. Responses are bounded in size and validated as closed Pydantic models. Provider errors are sanitized.

`json_schema: true` requests the compatible structured-output wire format; support varies by server. JSON-only mode still validates the returned object locally. An invalid response is rejected rather than repaired into executable code. The live-model audit documents the tested model and its narrow coverage; other models require their own evaluation.

7. Understand and modify the IR

The IR has version 1.0. Core fields describe pipeline identity, owners, schedule, timezone, start date, catchup, concurrency, retry/timeout policies, sources, destinations, tasks, quality, governance, notifications, lineage, resource limits, secrets, execution policy, and approval requirements.

Each task references a versioned capability such as `transform.deduplicate@1`. Inputs are `asset:orders` or `task:read_orders`; output tasks declare a destination such as `asset:analytics`. Dependencies reference task IDs. Quality rules target a task's output; the planner inserts a `q_TASK` barrier and redirects downstream consumers through it. Avoid manually creating IDs reserved for quality barriers.

```
id: deduplicate
type: transformation
capability: transform.deduplicate@1
inputs: [task:read_orders]
parameters:
  keys: [order_id]
```

The repository example contains the complete specification. The generated JSON Schema is in `docs/pipeline-ir.schema.json`. Unknown fields and IR versions are rejected; migrations require explicit implementation.

Modify an existing specification conversationally:

```
npipe create "Run it at 4 AM." --previous examples/orders.yaml \
  --catalog examples/catalog.yaml --out work/orders-v2.yaml
npipe diff examples/orders.yaml work/orders-v2.yaml
```

The offline frontend supports schedule, quarantine, and alert modifications. The model frontend can propose broader IR changes. Every change goes through validation. A prior approval never carries over to changed content. DRAFT, VALIDATED, APPROVED, and DEPLOYED are workflow states, not authority values a model may put in IR.

8. Quality, quarantine, retries, and notifications

Rules have explicit thresholds and actions. fail blocks downstream work when the invalid rate exceeds `max_invalid_rate`. quarantine writes invalid records to the configured quarantine asset and passes only valid rows downstream. The quality result can remain passed: false while the pipeline succeeds because quarantine handled the violation. Use a quality notification threshold to alert on the rate. Quarantine output is replaced on each run; retention and historical partitioning need an external lifecycle adapter.

Schema checks compare fields and primitive types. Datetime fields require timezone-aware ISO 8601 strings containing T; declared CSV boolean fields recognize lowercase true and false. Nullability is a separate `not_null` rule. After joins or schema-changing transforms, provide the expected schema explicitly in the schema rule's parameters. Freshness requires timezone-aware timestamps and an explicit maximum age in seconds. Range rules require numeric minimum and maximum bounds. No inferred threshold becomes authoritative.

Pipeline retry defaults are bounded (0-10 retries); task overrides are supported. Airflow enforces retry delay, exponential backoff, task execution timeout, pipeline timeout, and concurrency. The local runner is a functional verification path: it makes one attempt per task and detects exceeded time budgets after a task returns. It is not a hard-timeout scheduler. Use Airflow for production scheduling and cancellation.

Notifications write durable JSON events to a catalog-approved local outbox. They do not send email, Slack, or webhooks. An administrator may attach a delivery service. Airflow failure callbacks fire after the task/run failure semantics determined by Airflow. CPU and memory fields are planning metadata; actual worker isolation and quotas belong to the Airflow deployment. Row limits are enforced by the runtime.

9. Review, approve, and deploy

Validation produces PASS, WARN, FAIL, and REQUIRES_APPROVAL decisions. FAIL blocks compilation. REQUIRES_APPROVAL permits inspection and compilation but blocks execution and deployment until a valid approval exists. Production requires an owner. Production, destructive rebuilds, external writes, and approved schema evolution always require human review; model-supplied approval flags cannot disable these gates.

An administrator manages NLPIPE_APPROVAL_KEY, a signing secret of at least 32 characters. Store it outside the repository and IR. Give signing access only to approvers and verification access to the runtime. This shared-key MVP is not enterprise identity management; the actor name is an attributed label. Users who control the signing key can approve work.

```
nlpipe approve work/production.yaml --catalog examples/catalog.yaml \
  operator-name --out work/approval.json --hours 24
nlpipe deploy work/production.yaml --catalog examples/catalog.yaml \
  /path/to/airflow/dags --environment production \
  --approval work/approval.json
```

Set `execution_policy.environment`: production and a real accountable owner in the reviewed IR first. Approval binds normalized IR, catalog, environment, issue time, and expiry using HMAC. Changing the spec or catalog invalidates it. Deploy requires real Airflow import verification. The deployment manifest records the artifact hash and approval evidence.

Configure every Airflow worker with NLPIPE_CATALOG (absolute path to the trusted catalog), NLPIPE_DATA_ROOT (shared data/artifact root), and, when required, NLPIPE_APPROVAL_FILE plus NLPIPE_APPROVAL_KEY. Set NLPIPE_ALLOW_HTTP=1 only for catalog-approved HTTPS reads. Workers must share the artifact filesystem, and the installed package must match the tested release. Approval is checked again at task execution; an expired approval can block a scheduled run until renewed.

DEPLOYED means the artifact was installed in the selected DAG folder. It does not mean the scheduler loaded it or a run succeeded. Verify those separately through Airflow and imported observations. Restore a prior reviewed DAG and manifest to roll back; do not modify generated code by hand.

10. Observe and propose repairs

The normalized models include PipelineRun, TaskRun, RunStatus, FailureEvent, QualityResult, and LineageEvent. Local execution records task counts, durations, quality results, failure categories, skipped tasks, and lineage. Failure messages intentionally omit arbitrary data and credentials.

```
npipe status orders_daily work/demo-data
npipe ask orders_daily "Why did the last run fail?" work/demo-data
npipe observe examples/orders.yaml work/airflow-state.json \
  --out work/observation.json
npipe repair examples/orders.yaml work/failed-run.json \
  --out work/repair-proposal.json
```

observe consumes an administrator-exported Airflow JSON state containing dag_id, dag_run_id, state, and optional task instances. It does not poll an Airflow API. ask describes observed failures; it does not fabricate a root cause or performance trend. repair proposes one additional bounded retry after a failed run with a matching IR hash. It never automatically changes, approves, or deploys the pipeline.

The DataOps boundary is an interface: Observe, Assess, Recommend, propose an IR change, Validate, Approve, Repair, Verify. The existing agentic-data-ops project informed validation practices; this MVP does not import its domain-specific examples or claim a live integration. A future adapter can translate its evidence into proposed changes without bypassing approval.

11. Evaluate and test

```
pytest
pytest -m security
pytest -m airflow
pytest --cov=nlpipe --cov-report=term-missing
ruff check .
mypy src
nlpipe evaluate corpus/acceptance.json --catalog examples/catalog.yaml \
  --out work/evaluation
```

The corpus contains 50 requests, including the specified classes of simple, complex, conversational, ambiguous, conflicting, and unsafe requests. Twenty offline cases produce executable IR; thirty test clarification or rejection, including unavailable adapters. Accepted cases assert assets, capability sequence, dependency edges, schedules, quality behavior, policy, and selected canonical properties. This is not a claim that all requested integrations are implemented.

Results are saved as `eval-results.json` and `eval-report.md`. Each semantic dimension reports its own numerator and denominator; unscored dimensions do not count as passes. Run the same command with `--config` to compare a real model against the corpus. The mock provider is for boundary tests only and must never be reported as model accuracy.

Within each of three canonical pipeline groups, six paraphrases must normalize to equivalent offline IR. The optional live audit runs two paraphrases and default/nondefault explanation round-trips through an actual local model. Property tests cover dependency ordering and unknown capabilities. Security tests block network access and use controlled HTTP transports. Integration tests check exact output records and real Airflow imports; the release report distinguishes executed gates from configured future checks.

12. Troubleshooting and operational limits

Exit code 0 means the command completed its stated checks. Code 2 means invalid input, a denied operation, or a failed run/check. Code 3 means clarification is required. Always read the structured report: a command without `--airflow` has not tested Airflow, and a quarantined pipeline may succeed while recording failed quality thresholds.

If an asset is unresolved, check identifiers, aliases, permissions, and the selected catalog. If a capability is unavailable, implement and test an adapter; do not substitute an arbitrary Python operator. If approval fails, check exact spec/catalog hashes, environment, expiry, and the configured signing key. If workers cannot find artifacts, verify the shared root and identical catalog on every worker.

If a model returns malformed JSON or an incompatible schema, inspect its support for structured responses, context size, and timeout. Retry only after understanding the failure. No prompt can authorize raw generated Python, shell execution, plaintext credentials, or an unregistered connector.

The runtime is bounded, single-host/shared-filesystem oriented and not intended for large distributed datasets. Default row limits are 100,000; the absolute configurable ceiling is 1,000,000, with a 32 MiB serialized dataset/file boundary. Joins enforce the row limit during expansion. These bounds are safeguards, not a performance SLA. Use separate administrator-owned roots per tenant; this MVP is not a hostile multi-tenant sandbox. External side effects require adapter-specific retry/idempotency design.

Before production use, run representative synthetic and approved local fixtures, review the IR and catalog, validate the generated DAG in the actual Airflow environment, assign ownership, configure storage/retention and notification delivery, and monitor the first scheduled runs. See [VALIDATION.md](#), [SECURITY.md](#), and [LIMITATIONS.md](#) for measured release evidence and exact boundaries.